PYTHON ACTIVITY-BASED LEARNING

CASE STUDY 1 – UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement

Design a Python program to evaluate scholarship applicants using aggregate marks, subject-wise minimum scores, attendance, and family income. Eligible students are ranked using predefined tie-breaking rules.

2. Objectives

1. Check minimum marks in each subject.
2. Calculate aggregate percentage.
3. Validate attendance and annual family income.
4. Separate eligible and ineligible applicants.
5. Rank eligible applicants using aggregate, attendance, and alphabetical order.

3. Assumptions

• Three subjects are considered: Mathematics, Physics, and Computer Science; each is out of 100.

• Minimum mark in every subject is 50.

• Minimum aggregate is 70%.

• Minimum attendance is 75%.

• Annual family income must not exceed 2,50,000.

• Ties are resolved by higher aggregate, then higher attendance, then alphabetical name.

4. Algorithm

1. Store applicant details as tuples.
2. Set subject, aggregate, attendance, and income criteria.
3. Calculate each applicant's aggregate percentage.
4. Check every eligibility condition.
5. Store eligible and ineligible applicants separately.
6. Sort eligible applicants by the specified tie-breaking rules.
7. Display ranking and ineligible applicants.

5. Pseudocode

```
START
Set eligibility criteria
Read/store applicant details
FOR each applicant: calculate aggregate and check all conditions
IF all conditions pass, add to eligible list; otherwise add to ineligible list
Sort eligible list by aggregate, attendance, and name
Display both lists
STOP
```

6. Flowchart

START → Store applicant details → Calculate aggregate → Check subject marks, attendance, income and aggregate → Eligible? → YES: Eligible list / NO: Ineligible list → Rank eligible students → Display results → STOP

7. Python Source Code

```
# University Scholarship Allocation

minimum_subject_mark = 50
minimum_aggregate = 70
minimum_attendance = 75
maximum_income = 250000

# (Name, Maths, Physics, Computer Science, Attendance, Annual Income)
applicants = [
    ("Anil", 82, 76, 88, 92, 180000),
    ("Bhavya", 75, 72, 78, 85, 220000),
    ("Charan", 74, 70, 76, 80, 200000),
```

```
    ("Divya", 65, 60, 70, 90, 190000),
    ("Esha", 90, 85, 88, 70, 150000),
    ("Farhan", 75, 72, 78, 85, 220000)
]

eligible = []
ineligible = []

for student in applicants:
    name, maths, physics, computer, attendance, income = student
    aggregate = (maths + physics + computer) / 3

    if (maths >= minimum_subject_mark and
        physics >= minimum_subject_mark and
        computer >= minimum_subject_mark and
        aggregate >= minimum_aggregate and
        attendance >= minimum_attendance and
        income <= maximum_income):
        eligible.append((name, maths, physics, computer,
                         attendance, income, aggregate))
    else:
        ineligible.append((name, aggregate, attendance, income))

# Higher aggregate first, then attendance, then alphabetical name
for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        first = eligible[i]
        second = eligible[j]
        if (first[6] < second[6] or
            (first[6] == second[6] and first[4] < second[4]) or
            (first[6] == second[6] and first[4] == second[4]
             and first[0] > second[0])):
            eligible[i], eligible[j] = eligible[j], eligible[i]

print("UNIVERSITY SCHOLARSHIP RANKING")
print("------------------------------")
for rank, student in enumerate(eligible, start=1):
    name, maths, physics, computer, attendance, income, aggregate = student
    print("Rank:", rank)
    print("Name:", name)
    print("Aggregate:", round(aggregate, 2), "%")
    print("Attendance:", attendance, "%")
    print("Annual Income: Rs.", income)
    print("------------------------------")

print("INELIGIBLE APPLICANTS")
print("------------------------------")
for student in ineligible:
    name, aggregate, attendance, income = student
    print("Name:", name)
    print("Aggregate:", round(aggregate, 2), "%")
    print("Attendance:", attendance, "%")
    print("Annual Income: Rs.", income)
    print("------------------------------")
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	Anil	Meets all conditions; high aggregate	Eligible
2	Bhavya	Meets all conditions	Eligible
3	Charan	Meets all conditions	Eligible
4	Divya	Aggregate below 70%	Ineligible
5	Esha	Attendance below 75%	Ineligible
6	Farhan	Same marks and as Bhavya alphabetical tie-break	Eligiblble

9. Sample Output

```
UNIVERSITY SCHOLARSHIP RANKING
------------------------------
Rank: 1
Name: Anil
Aggregate: 82.0 %
Attendance: 92 %
Annual Income: Rs. 180000
------------------------------
Rank: 2
Name: Bhavya
Aggregate: 75.0 %
Attendance: 85 %
```

```
Annual Income: Rs. 220000
------------------------------
Rank: 3
Name: Farhan
Aggregate: 75.0 %
Attendance: 85 %
Annual Income: Rs. 220000
------------------------------
Rank: 4
Name: Charan
Aggregate: 73.33 %
Attendance: 80 %
Annual Income: Rs. 200000
------------------------------
INELIGIBLE APPLICANTS
------------------------------
Name: Divya
Aggregate: 65.0 %
Attendance: 90 %
Annual Income: Rs. 190000
------------------------------
Name: Esha
Aggregate: 87.67 %
Attendance: 70 %
Annual Income: Rs. 150000
------------------------------
```

10. Conclusion

The system checks academic performance, attendance, and family income before ranking eligible scholarship applicants. It demonstrates tuples, lists, conditions, loops, aggregate calculations, and tie-breaking logic.

CASE STUDY 2 – HOSPITAL BILLING AND PATIENT CLASSIFICATION

1. Problem Statement

Design a Python program that calculates a patient's hospital bill from consultation, diagnostic, and treatment charges, applies concessions according to a payment policy, validates billing constraints, and classifies the patient by the final payable amount.

2. Objectives

1. Calculate total charges.

2. Validate that charges and payment are non-negative.

3. Apply a concession when the stated condition is met.

4. Calculate final payable amount and balance due.

5. Classify the bill according to the supplied policy.

3. Assumptions

• All charges are in rupees and must be non-negative.

• A 10% concession is applied when the total bill is at least 10,000.

• Payment cannot be negative or greater than the final bill.

• Final payable amount below 5,000 is Basic, 5,000–4,999 is Standard, and 15,000 or above is High.

4. Algorithm

1. Store patient and charge details.

2. Validate each charge and amount paid.

3. Add consultation, diagnostic, and treatment charges.

4. Apply the concession if the total reaches the threshold.

5. Calculate final bill and balance due.

6. Classify the patient using final bill amount.

7. Display the bill or an invalid-input message.

5. Pseudocode

```
START
Read patient name, charges and amount paid
IF any input is negative, display invalid input
Calculate total charges
IF total is at least 10,000, calculate 10% concession
Calculate final bill and check payment constraint
Classify patient and display bill details
STOP
```

6. Flowchart

START → Input charges and payment → Validate inputs → Calculate total → Apply concession if eligible → Calculate final bill → Validate payment → Classify bill → Display invoice → STOP

7. Python Source Code

```
# Hospital Billing and Patient Classification

patient_name = "Ravi"
consultation = 800
diagnostic = 4200
treatment = 6000
amount_paid = 5000

if (consultation < 0 or diagnostic < 0 or treatment < 0
        or amount_paid < 0):
    print("Invalid input: charges and payment cannot be negative.")
else:
```

```
total = consultation + diagnostic + treatment

if total >= 10000:
    concession = total * 0.10
else:
    concession = 0

final_bill = total - concession

if amount_paid > final_bill:
    print("Invalid payment: payment exceeds final bill.")
else:
    balance = final_bill - amount_paid

    if final_bill < 5000:
        category = "Basic"
    elif final_bill < 15000:
        category = "Standard"
    else:
        category = "High"

    print("HOSPITAL BILL")
    print("-------------")
    print("Patient:", patient_name)
    print("Consultation Charges: Rs.", consultation)
    print("Diagnostic Charges: Rs.", diagnostic)
    print("Treatment Charges: Rs.", treatment)
    print("Total Charges: Rs.", round(total, 2))
    print("Concession: Rs.", round(concession, 2))
    print("Final Bill: Rs.", round(final_bill, 2))
    print("Amount Paid: Rs.", amount_paid)
    print("Balance Due: Rs.", round(balance, 2))
    print("Patient Classification:", category)
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	Ravi	Total ■11,000; payment ■5,000	■1,100 concession; Standard
2	Maya	Total below ■10,000	No concession
3	Kiran	Negative diagnostic charge	Invalid input
4	Sita	Payment greater than final bill	Invalid payment

9. Sample Output

```
HOSPITAL BILL
-------------
Patient: Ravi
Consultation Charges: Rs. 800
Diagnostic Charges: Rs. 4200
Treatment Charges: Rs. 6000
Total Charges: Rs. 11000
Concession: Rs. 1100.0
Final Bill: Rs. 9900.0
Amount Paid: Rs. 5000
Balance Due: Rs. 4900.0
Patient Classification: Standard
```

10. Conclusion

The program combines hospital charges, applies a conditional concession, validates payments, and classifies the final bill. It demonstrates arithmetic operations, if-elif-else conditions, and input validation.

CASE STUDY 3 – EXAMINATION RESULT AND GRACE-MARK VALIDATION

1. Problem Statement

Design a Python program to process marks in multiple subjects, apply subject-wise pass criteria and a limited grace-mark policy, and determine the final result without allowing marks to exceed the maximum.

2. Objectives

1. Validate marks against the permitted range.
2. Check pass marks for every subject.
3. Apply grace marks only under the defined rule.
4. Ensure moderated marks do not exceed maximum marks.
5. Display original marks, final marks, and result.

3. Assumptions

• Three subjects are considered, each out of 100.

• Pass mark is 40 in each subject.

• At most 5 grace marks may be added to one subject to help a student reach 40.

• Grace marks are used only if the student has failed exactly one subject and needs no more than 5 marks.

• Final marks cannot exceed 100.

4. Algorithm

1. Store marks for each subject.
2. Validate each mark is between 0 and 100.
3. Identify subjects below 40.
4. If exactly one subject is failed and the gap is at most 5, add the required grace marks.
5. Cap final marks at 100.
6. Check all final subject marks and print PASS or FAIL.

5. Pseudocode

```
START
Read marks for three subjects
Validate marks between 0 and 100
Find subjects below pass mark
IF exactly one subject failed and required grace is at most 5, add grace marks
Ensure final marks do not exceed 100
Check all subjects and display result
STOP
```

6. Flowchart

START → Input marks → Validate range → Find failed subjects → Grace policy satisfied? → YES: Apply required grace / NO: Keep marks unchanged → Check final pass criteria → Display result → STOP

7. Python Source Code

```
# Examination Result and Grace-Mark Validation

subject_names = ["Mathematics", "Physics", "Computer Science"]
original_marks = [38, 65, 72]
pass_mark = 40
maximum_mark = 100
maximum_grace = 5

valid = True
for mark in original_marks:
    if mark < 0 or mark > maximum_mark:
        valid = False

if not valid:
```

```
    print("Invalid marks: each mark must be between 0 and 100.")
else:
    final_marks = original_marks.copy()
    failed = []

    for i in range(len(original_marks)):
        if original_marks[i] < pass_mark:
            failed.append(i)

    grace_used = 0
    if len(failed) == 1:
        index = failed[0]
        required = pass_mark - final_marks[index]
        if required <= maximum_grace:
            final_marks[index] = min(maximum_mark,
                                     final_marks[index] + required)
            grace_used = required

    print("EXAMINATION RESULT")
    print("------------------")
    for i in range(len(subject_names)):
        print(subject_names[i] + ":",
              original_marks[i], "->", final_marks[i])

    print("Grace Marks Used:", grace_used)
    if all(mark >= pass_mark for mark in final_marks):
        print("Final Result: PASS")
    else:
        print("Final Result: FAIL")
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	38, 65, 72	One subject short by 2 marks	PASS after 2 grace marks
2	32, 65, 72	Gap is greater than 5	FAIL
3	45, 65, 72	All subjects passed	PASS; no grace
4	101, 65, 72	Mark exceeds maximum	Invalid marks

9. Sample Output

```
EXAMINATION RESULT
------------------
Mathematics: 38 -> 40
Physics: 65 -> 65
Computer Science: 72 -> 72
Grace Marks Used: 2
Final Result: PASS
```

10. Conclusion

The program validates marks, identifies failed subjects, applies a limited grace policy, and prevents marks from exceeding the maximum. It demonstrates lists, loops, conditions, and the min() function.

CASE STUDY 4 – ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement

Design a Python program to simulate a sequence of ATM deposits and withdrawals, maintain the account balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

2. Objectives

1. Maintain a running balance.
2. Process deposits and withdrawals in sequence.
3. Reject non-positive transactions.
4. Reject withdrawals exceeding balance or daily limit.
5. Display accepted and rejected transactions and final balance.

3. Assumptions

• Opening balance is ■5,000.

• A transaction amount must be greater than zero.

• Maximum withdrawal per transaction is ■3,000.

• Total withdrawals in this simulation cannot exceed ■5,000.

• Deposits have no upper limit in this exercise.

4. Algorithm

1. Set opening balance and withdrawal counters.
2. Store transactions as type and amount pairs.
3. For each transaction, validate amount and type.
4. Accept valid deposits and update balance.
5. For withdrawals, check transaction limit and available balance and cumulative limit.
6. Record accepted or rejected status.
7. Display transaction summary and closing balance.

5. Pseudocode

```
START
Set opening balance and withdrawal limits
FOR each transaction
Validate type and positive amount
IF deposit, add amount to balance
ELSE IF withdrawal meets limits and balance, subtract amount
Otherwise reject transaction
Display statuses and closing balance
STOP
```

6. Flowchart

START → Set opening balance → Read transaction sequence → Validate transaction → Deposit or withdrawal? → Check applicable limits → Update balance or reject → Continue until sequence ends → Summary → STOP

7. Python Source Code

```
# ATM Transaction Sequence Simulation

balance = 5000
maximum_withdrawal = 3000
daily_withdrawal_limit = 5000
total_withdrawn = 0

transactions = [
    ("deposit", 2000),
    ("withdraw", 1500),
    ("withdraw", 4000),
```

```
    ("deposit", 0),
    ("withdraw", 2500)
]

accepted = 0
rejected = 0

print("ATM TRANSACTION SUMMARY")
print("-----------------------")
print("Opening Balance: Rs.", balance)

for transaction_type, amount in transactions:
    status = "Rejected"

    if amount <= 0:
        status = "Rejected - amount must be positive"
        rejected += 1
    elif transaction_type == "deposit":
        balance += amount
        status = "Accepted"
        accepted += 1
    elif transaction_type == "withdraw":
        if amount > maximum_withdrawal:
            status = "Rejected - per-transaction limit exceeded"
            rejected += 1
        elif total_withdrawn + amount > daily_withdrawal_limit:
            status = "Rejected - daily withdrawal limit exceeded"
            rejected += 1
        elif amount > balance:
            status = "Rejected - insufficient balance"
            rejected += 1
        else:
            balance -= amount
            total_withdrawn += amount
            status = "Accepted"
            accepted += 1
    else:
        status = "Rejected - invalid transaction type"
        rejected += 1

    print(transaction_type.title(), "Rs.", amount, ":", status)
    print("Balance: Rs.", balance)

print("-----------------------")
print("Accepted Transactions:", accepted)
print("Rejected Transactions:", rejected)
print("Total Withdrawn: Rs.", total_withdrawn)
print("Closing Balance: Rs.", balance)
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	deposit ■2,000	Positive deposit	Accepted
2	withdraw ■1,500	Within limits and balance	Accepted
3	withdraw ■4,000	Exceeds per-transaction limit	Rejected
4	deposit ■0	Non-positive amount	Rejected
5	withdraw ■2,500	Within limits and balance	Accepted

9. Sample Output

```
ATM TRANSACTION SUMMARY
-----------------------
Opening Balance: Rs. 5000
Deposit Rs. 2000 : Accepted
Balance: Rs. 7000
Withdraw Rs. 1500 : Accepted
Balance: Rs. 5500
Withdraw Rs. 4000 : Rejected - per-transaction limit exceeded
Balance: Rs. 5500
Deposit Rs. 0 : Rejected - amount must be positive
Balance: Rs. 5500
Withdraw Rs. 2500 : Accepted
Balance: Rs. 3000
-----------------------
Accepted Transactions: 3
Rejected Transactions: 2
Total Withdrawn: Rs. 4000
Closing Balance: Rs. 3000
```

10. Conclusion

The simulation updates the account balance only for valid transactions and enforces withdrawal rules. It demonstrates loops, conditions, counters, and state updates.

CASE STUDY 5 – SMART PARKING FEE MANAGEMENT

1. Problem Statement

Design a Python program to calculate parking fees from entry and exit times, vehicle category, overnight parking, daily limits, and penalty conditions while validating time inputs.

2. Objectives

1. Validate entry and exit times.

2. Calculate parking duration including overnight parking.

3. Apply vehicle-specific hourly rates.

4. Apply daily fee limits.

5. Add a penalty when the vehicle stays beyond the stated period.

3. Assumptions

• Time is entered as 24-hour hour-and-minute values.

• If exit time is earlier than entry time, the vehicle exited the next day.

• Any partial hour is charged as a full hour.

• Car rate is ■40 per hour; bike rate is ■20 per hour.

• Maximum normal charge is ■400 per 24-hour period.

• A ■200 penalty applies when parking exceeds 48 hours.

• This simplified model handles at most one overnight rollover.

4. Algorithm

1. Read entry time, exit time, and vehicle category.

2. Validate hours, minutes, and category.

3. Convert times to minutes and calculate duration, adding one day if exit is earlier.

4. Round duration up to a whole number of hours.

5. Select hourly rate based on vehicle category.

6. Calculate charge using daily cap and add penalty if duration exceeds 48 hours.

7. Display duration and final fee.

5. Pseudocode

```
START
Input entry/exit time and vehicle category
Validate time and category
Convert times to minutes
IF exit earlier than entry, add 24 hours
Calculate duration and billable hours
Apply rate and daily cap; add penalty if applicable
Display parking fee
STOP
```

6. Flowchart

START → Input times and vehicle type → Validate → Calculate duration → Determine billable hours → Apply vehicle rate and daily cap → Check penalty → Display fee → STOP

7. Python Source Code

```
# Smart Parking Fee Management
import math

entry_hour = 22
entry_minute = 30
exit_hour = 2
exit_minute = 15
vehicle = "car"
```

```
if not (0 <= entry_hour <= 23 and 0 <= exit_hour <= 23 and
        0 <= entry_minute <= 59 and 0 <= exit_minute <= 59):
    print("Invalid time input.")
elif vehicle not in ["car", "bike"]:
    print("Invalid vehicle category.")
else:
    entry = entry_hour * 60 + entry_minute
    exit_time = exit_hour * 60 + exit_minute

    if exit_time < entry:
        exit_time += 24 * 60

    duration_minutes = exit_time - entry
    hours = math.ceil(duration_minutes / 60)

    if vehicle == "car":
        hourly_rate = 40
    else:
        hourly_rate = 20

    # Cap each started 24-hour block at Rs. 400
    days = math.ceil(duration_minutes / (24 * 60))
    raw_charge = hours * hourly_rate
    fee = min(raw_charge, days * 400)

    penalty = 200 if duration_minutes > 48 * 60 else 0
    final_fee = fee + penalty

    print("SMART PARKING BILL")
    print("------------------")
    print("Vehicle:", vehicle.title())
    print("Duration:", duration_minutes // 60, "hours and",
          duration_minutes % 60, "minutes")
    print("Billable Hours:", hours)
    print("Parking Charge: Rs.", fee)
    print("Penalty: Rs.", penalty)
    print("Total Fee: Rs.", final_fee)
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	22:30 to 02:15, car	Overnight duration 3h 45m	4 hours × ■40 = ■160
2	09:00 to 10:05, bike	Partial hour rounded up	2 hours × ■20 = ■40
3	Invalid hour 25	Time outside 0–23	Invalid time input
4	Stay exceeds 48 hours	Penalty condition applies	Add ■200 penalty

9. Sample Output

```
SMART PARKING BILL
------------------
Vehicle: Car
Duration: 3 hours and 45 minutes
Billable Hours: 4
Parking Charge: Rs. 160
Penalty: Rs. 0
Total Fee: Rs. 160
```

10. Conclusion

The program converts clock times into minutes, accounts for a simple overnight stay, rounds partial hours upward, and calculates fees using vehicle rates and limits. It demonstrates arithmetic, validation, conditions, and the math module.

CASE STUDY 6 – MULTI-STAGE SCHOLARSHIP SHORTLISTING

1. Problem Statement

Design a Python program that applies minimum academic eligibility, attendance requirements, income conditions, and ranking rules to select candidates within a fixed scholarship quota.

2. Objectives

1. Apply academic eligibility rules.
2. Check attendance and family income.
3. Separate eligible and ineligible candidates.
4. Rank eligible candidates consistently.
5. Select only the number allowed by the scholarship quota.

3. Assumptions

• Minimum aggregate is 75%.

• Minimum attendance is 80%.

• Annual family income must be at most ■3,00,000.

• Scholarship quota is 3 students.

• Candidates are ranked by higher aggregate, then higher attendance, then alphabetical name.

4. Algorithm

1. Store candidate records.
2. Check aggregate, attendance, and income requirements.
3. Add candidates to eligible or ineligible lists.
4. Sort eligible candidates using the ranking rules.
5. Select the first candidates up to the quota.
6. Display selected, waitlisted, and ineligible candidates.

5. Pseudocode

```
START
Set minimum criteria and quota
Check every candidate's eligibility
Separate eligible and ineligible lists
Sort eligible candidates by ranking rules
Select first quota candidates; remaining candidates are waitlisted
Display all categories
STOP
```

6. Flowchart

START → Input candidates → Check academic, attendance and income criteria → Eligible? → Rank eligible list → Apply quota → Selected / Waitlisted / Ineligible → Display → STOP

7. Python Source Code

```
# Multi-Stage Scholarship Shortlisting

minimum_aggregate = 75
minimum_attendance = 80
maximum_income = 300000
quota = 3

# (Name, Aggregate, Attendance, Annual Income)
candidates = [
    ("Anil", 88, 92, 200000),
    ("Bhavya", 81, 85, 250000),
    ("Charan", 79, 88, 280000),
    ("Divya", 74, 95, 180000),
    ("Esha", 85, 78, 190000),
```

```
    ("Farhan", 81, 85, 250000)
]

eligible = []
ineligible = []

for candidate in candidates:
    name, aggregate, attendance, income = candidate
    if (aggregate >= minimum_aggregate and
        attendance >= minimum_attendance and
        income <= maximum_income):
        eligible.append(candidate)
    else:
        ineligible.append(candidate)

for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        a = eligible[i]
        b = eligible[j]
        if (a[1] < b[1] or
            (a[1] == b[1] and a[2] < b[2]) or
            (a[1] == b[1] and a[2] == b[2] and a[0] > b[0])):
            eligible[i], eligible[j] = eligible[j], eligible[i]

selected = eligible[:quota]
waitlisted = eligible[quota:]

print("SCHOLARSHIP SELECTED CANDIDATES")
print("--------------------------------")
for rank, candidate in enumerate(selected, start=1):
    print("Rank:", rank, "| Name:", candidate[0],
          "| Aggregate:", candidate[1], "%",
          "| Attendance:", candidate[2], "%")

print("\nWAITLISTED CANDIDATES")
print("--------------------------------")
for candidate in waitlisted:
    print("Name:", candidate[0], "| Aggregate:", candidate[1], "%")

print("\nINELIGIBLE CANDIDATES")
print("--------------------------------")
for candidate in ineligible:
    print("Name:", candidate[0], "| Aggregate:", candidate[1], "%",
          "| Attendance:", candidate[2], "%")
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	Anil	Meets all criteria; top aggregate	Selected
2	Bhavya	Meets criteria	Selected
3	Charan	Meets criteria	Selected if within quota
4	Divya	Aggregate below 75%	Ineligible
5	Esha	Attendance below 80%	Ineligible
6	Farhan	Tied with Bhavya; alphabetical rule	Waitlisted if quota is full

9. Sample Output

```
SCHOLARSHIP SELECTED CANDIDATES
--------------------------------
Rank: 1 | Name: Anil | Aggregate: 88 % | Attendance: 92 %
Rank: 2 | Name: Bhavya | Aggregate: 81 % | Attendance: 85 %
Rank: 3 | Name: Farhan | Aggregate: 81 % | Attendance: 85 %

WAITLISTED CANDIDATES
--------------------------------
Name: Charan | Aggregate: 79 %

INELIGIBLE CANDIDATES
--------------------------------
Name: Divya | Aggregate: 74 % | Attendance: 95 %
Name: Esha | Aggregate: 85 % | Attendance: 78 %
```

10. Conclusion

The program performs multiple eligibility checks, ranks qualified applicants, and enforces a fixed scholarship quota. It demonstrates lists, tuples, nested loops, slicing, and conditional logic.

CASE STUDY 7 – SMART HOSTEL RESOURCE MONITORING

1. Problem Statement

Design a Python program to analyse room-wise electricity and water consumption, calculate charges, identify threshold violations, and compare resource utilization across hostel floors.

2. Objectives

1. Calculate electricity and water charges for each room.
2. Identify consumption above defined limits.
3. Calculate total consumption and cost floor-wise.
4. Compare resource usage across floors.
5. Display room-level and floor-level summaries.

3. Assumptions

• Electricity costs ■8 per unit.

• Water costs ■20 per kilolitre.

• Electricity threshold is 150 units per room.

• Water threshold is 5 kilolitres per room.

• A room violates a threshold if either resource exceeds its limit.

• Each record contains room number, floor, electricity units, and water kilolitres.

4. Algorithm

1. Store room consumption records.
2. Calculate electricity, water, and total charges for each room.
3. Check resource thresholds.
4. Accumulate consumption and charges by floor.
5. Display room-level results and floor totals.
6. Identify the floor with the highest electricity and water use.

5. Pseudocode

```
START
Set rates and thresholds
Read room-wise consumption
FOR each room: calculate charges and check thresholds
Add values to floor totals
Display room details and violations
Compare floor totals and display highest usage
STOP
```

6. Flowchart

START → Store room readings → Calculate charges → Check thresholds → Accumulate floor totals → Display room violations → Compare floor consumption → STOP

7. Python Source Code

```
# Smart Hostel Resource Monitoring

electricity_rate = 8
water_rate = 20
electricity_limit = 150
water_limit = 5

# (Room, Floor, Electricity Units, Water Kilolitres)
rooms = [
    ("101", 1, 120, 4),
    ("102", 1, 180, 6),
    ("201", 2, 140, 3),
    ("202", 2, 160, 7),
    ("301", 3, 100, 4)
```

```
]

floor_totals = {}

print("HOSTEL ROOM RESOURCE REPORT")
print("---------------------------")
for room, floor, electricity, water in rooms:
    electricity_charge = electricity * electricity_rate
    water_charge = water * water_rate
    total_charge = electricity_charge + water_charge

    violations = []
    if electricity > electricity_limit:
        violations.append("Electricity")
    if water > water_limit:
        violations.append("Water")

    if floor not in floor_totals:
        floor_totals[floor] = [0, 0, 0]

    floor_totals[floor][0] += electricity
    floor_totals[floor][1] += water
    floor_totals[floor][2] += total_charge

    print("Room:", room, "| Floor:", floor)
    print("Electricity:", electricity, "units")
    print("Water:", water, "kilolitres")
    print("Total Charge: Rs.", total_charge)
    print("Threshold Status:",
          ", ".join(violations) if violations else "Within limits")
    print("---------------------------")

print("FLOOR-WISE UTILIZATION")
for floor in sorted(floor_totals):
    values = floor_totals[floor]
    print("Floor", floor, "| Electricity:", values[0],
          "| Water:", values[1], "| Charges: Rs.", values[2])

highest_electricity = max(floor_totals,
                          key=lambda f: floor_totals[f][0])
highest_water = max(floor_totals,
                    key=lambda f: floor_totals[f][1])
print("Highest Electricity Use: Floor", highest_electricity)
print("Highest Water Use: Floor", highest_water)
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	Room 101: 120 units, 4 kL	Both below thresholds	Within limits
2	Room 102: 180 units, 6 kL	Both exceed thresholds	Electricity and Water violation
3	Room 202: 160 units, 7 kL	Both exceed thresholds	Electricity and Water violation
4	Compare floor totals	Aggregate by floor	Floor 1 highest electricity; Floor 2 highest water

9. Sample Output

```
HOSTEL ROOM RESOURCE REPORT
---------------------------
Room: 101 | Floor: 1
Electricity: 120 units
Water: 4 kilolitres
Total Charge: Rs. 1040
Threshold Status: Within limits
---------------------------
Room: 102 | Floor: 1
Electricity: 180 units
Water: 6 kilolitres
Total Charge: Rs. 1560
Threshold Status: Electricity, Water
---------------------------
Room: 201 | Floor: 2
Electricity: 140 units
Water: 3 kilolitres
Total Charge: Rs. 1180
Threshold Status: Within limits
---------------------------
Room: 202 | Floor: 2
Electricity: 160 units
Water: 7 kilolitres
Total Charge: Rs. 1420
Threshold Status: Electricity, Water
```

```
----------------------------
Room: 301 | Floor: 3
Electricity: 100 units
Water: 4 kilolitres
Total Charge: Rs. 880
Threshold Status: Within limits
----------------------------
FLOOR-WISE UTILIZATION
Floor 1 | Electricity: 300 | Water: 10 | Charges: Rs. 2600
Floor 2 | Electricity: 300 | Water: 10 | Charges: Rs. 2600
Floor 3 | Electricity: 100 | Water: 4 | Charges: Rs. 880
Highest Electricity Use: Floor 1
Highest Water Use: Floor 1
```

10. Conclusion

The program computes room charges, checks resource thresholds, and aggregates use by floor. It demonstrates dictionaries, lists, loops, conditions, and max() for comparisons.

CASE STUDY 8 – UNIVERSITY EXAMINATION RESULT MODERATION

1. Problem Statement

Design a Python program to process marks across multiple subjects, apply specified moderation rules, preserve maximum-mark constraints, and compare original and moderated outcomes.

2. Objectives

1. Validate original marks.

2. Apply a defined moderation rule.

3. Prevent marks from exceeding the subject maximum.

4. Compare original and moderated totals and results.

5. Display the change caused by moderation.

3. Assumptions

• Three subjects are considered, each out of 100.

• Pass mark is 40 per subject.

• A student may receive 3 moderation marks in total, added to failed subjects in order, only as needed to reach 40.

• Moderation cannot increase a subject above 100.

• The result is PASS only when all moderated subject marks are at least 40.

4. Algorithm

1. Store subject names and original marks.

2. Validate marks from 0 to 100.

3. Copy original marks for moderation.

4. Calculate the total marks needed to pass failed subjects.

5. Apply up to 3 moderation marks in subject order, only as needed.

6. Compare original and moderated totals and classifications.

7. Display both results.

5. Pseudocode

```
START
Input subject marks
Validate every mark
Copy original marks
FOR each failed subject, add needed marks within remaining moderation limit
Do not exceed maximum marks
Determine original and moderated results
Display comparison
STOP
```

6. Flowchart

START → Input marks → Validate → Determine original result → Apply moderation allowance → Enforce pass and maximum constraints → Determine moderated result → Compare and display → STOP

7. Python Source Code

```
# University Examination Result Moderation

subjects = ["Mathematics", "Physics", "Computer Science"]
original_marks = [38, 39, 70]
pass_mark = 40
maximum_mark = 100
moderation_limit = 3

if any(mark < 0 or mark > maximum_mark for mark in original_marks):
    print("Invalid marks: marks must be between 0 and 100.")
else:
```

```
moderated_marks = original_marks.copy()
remaining = moderation_limit

for i in range(len(moderated_marks)):
    if moderated_marks[i] < pass_mark and remaining > 0:
        needed = pass_mark - moderated_marks[i]
        added = min(needed, remaining,
                    maximum_mark - moderated_marks[i])
        moderated_marks[i] += added
        remaining -= added

original_total = sum(original_marks)
moderated_total = sum(moderated_marks)

original_result = ("PASS" if all(
    mark >= pass_mark for mark in original_marks) else "FAIL")
moderated_result = ("PASS" if all(
    mark >= pass_mark for mark in moderated_marks) else "FAIL")

print("EXAMINATION MODERATION REPORT")
print("-----------------------------")
for i in range(len(subjects)):
    print(subjects[i], "Original:", original_marks[i],
          "| Moderated:", moderated_marks[i])
print("Original Total:", original_total)
print("Moderated Total:", moderated_total)
print("Original Result:", original_result)
print("Moderated Result:", moderated_result)
print("Moderation Marks Used:",
      moderated_total - original_total)
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	38, 39, 70	Needs 2 + 1 moderation marks	Moderated PASS
2	30, 39, 70	Needs 10 marks in first subject	Moderation limit prevents pass
3	40, 55, 60	Already passed	No moderation needed
4	100, 101, 60	One mark exceeds maximum	Invalid marks

9. Sample Output

```
EXAMINATION MODERATION REPORT
-----------------------------
Mathematics Original: 38 | Moderated: 40
Physics Original: 39 | Moderated: 40
Computer Science Original: 70 | Moderated: 70
Original Total: 147
Moderated Total: 150
Original Result: FAIL
Moderated Result: PASS
Moderation Marks Used: 3
```

10. Conclusion

The program compares original and moderated results while enforcing both the moderation limit and maximum marks. It demonstrates list copying, loops, min(), sum(), and all().

CASE STUDY 9 – EXAMINATION QUESTION DIFFICULTY AND PERFORMANCE ANALYSIS

1. Problem Statement

Design a Python program to compare student answer strings with an answer key, calculate question-wise success rates, identify the most frequently missed questions, and classify question performance.

2. Objectives

1. Compare each answer with the corresponding key.

2. Count correct answers for every question.

3. Calculate question-wise success rates.

4. Identify the most frequently missed question.

5. Classify questions by their success rate.

3. Assumptions

• All answer strings have the same length as the key.

• Each position represents one question.

• Answers are case-insensitive; a blank or incorrect answer is counted as incorrect.

• Success rate of 80% or more is Easy/High Success; 50% to below 80% is Moderate; below 50% is Difficult/Low Success.

• If multiple questions share the lowest success rate, all are reported.

4. Algorithm

1. Store the answer key and student answer strings.

2. Validate that each answer string has the correct length.

3. Initialize a correct counter for each question.

4. Compare every student's answer with the key position.

5. Calculate success percentage for each question.

6. Classify question performance and identify lowest success rate.

7. Display question-wise analysis.

5. Pseudocode

```
START
Store answer key and student responses
Validate response lengths
FOR each question position, count correct responses
Calculate success percentage
Classify success rate
Identify question(s) with lowest success rate
Display report
STOP
```

6. Flowchart

START → Store key and responses → Validate lengths → Compare answers question-wise → Calculate success rates → Classify difficulty → Find most missed questions → Display report → STOP

7. Python Source Code

```
# Examination Question Difficulty and Performance Analysis

answer_key = "ABCD"
student_answers = [
    "ABCD",
    "ABAD",
    "BBCD",
    "ABCC",
    "ABCD"
```

```
]

question_count = len(answer_key)
valid = True

for answers in student_answers:
    if len(answers) != question_count:
        valid = False

if not valid:
    print("Invalid input: every response must match key length.")
else:
    correct_counts = [0] * question_count

    for answers in student_answers:
        answers = answers.upper()
        for i in range(question_count):
            if answers[i] == answer_key[i]:
                correct_counts[i] += 1

    rates = []
    print("QUESTION PERFORMANCE ANALYSIS")
    print("------------------------------")
    for i in range(question_count):
        rate = correct_counts[i] / len(student_answers) * 100
        rates.append(rate)

        if rate >= 80:
            category = "Easy / High Success"
        elif rate >= 50:
            category = "Moderate"
        else:
            category = "Difficult / Low Success"

        missed = len(student_answers) - correct_counts[i]
        print("Question", i + 1)
        print("Correct:", correct_counts[i])
        print("Missed:", missed)
        print("Success Rate:", round(rate, 2), "%")
        print("Classification:", category)
        print("------------------------------")

    lowest_rate = min(rates)
    print("Most Frequently Missed Question(s):", end=" ")
    for i in range(question_count):
        if rates[i] == lowest_rate:
            print(i + 1, end=" ")
    print()
    print("Lowest Success Rate:", round(lowest_rate, 2), "%")
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	Key AB; answers AB, AB	Both responses correct	100% success
2	Question answered correctly by 2 of 5	40% success	Difficult / Low Success
3	Response length differs from key	Invalid format	Rejected
4	Two questions share lowest rate	Tie for most missed	Report both

9. Sample Output

```
QUESTION PERFORMANCE ANALYSIS
------------------------------
Question 1
Correct: 4
Missed: 1
Success Rate: 80.0 %
Classification: Easy / High Success
------------------------------
Question 2
Correct: 5
Missed: 0
Success Rate: 100.0 %
Classification: Easy / High Success
------------------------------
Question 3
Correct: 4
Missed: 1
Success Rate: 80.0 %
Classification: Easy / High Success
------------------------------
Question 4
```

```
Correct: 4
Missed: 1
Success Rate: 80.0 %
Classification: Easy / High Success
-----------------------------
Most Frequently Missed Question(s): 1 3 4
Lowest Success Rate: 80.0 %
```

10. Conclusion

The program analyses responses position by position, calculates success rates, and identifies questions with the lowest success. It demonstrates nested loops, lists, string indexing, and percentage calculations.

CASE STUDY 10 – UNIVERSITY HOSTEL ROOM ALLOCATION

1. Problem Statement

Design a Python program to allocate students to hostel rooms according to room capacity, student eligibility, and grouping constraints. The program must detect duplicate assignments and calculate unallocated demand.

2. Objectives

1. Check student eligibility for hostel accommodation.
2. Allocate students only when room capacity is available.
3. Respect room eligibility and grouping constraints.
4. Detect duplicate student assignments.
5. Calculate unallocated students and remaining room capacity.

3. Assumptions

• A student is eligible only if hostel approval is True.

• Rooms have a fixed capacity and accept only the specified category (for this example, gender category).

• Each student can be assigned to one room only.

• Students are processed in the order listed.

• Students who cannot be allocated are reported as unallocated.

4. Algorithm

1. Store student details and room capacities/categories.
2. Create an empty allocation list for each room.
3. Process each student.
4. Reject ineligible students.
5. Check whether the student is already assigned.
6. Find the first room with matching category and available capacity.
7. Allocate or mark the student as unallocated.
8. Display room assignments, duplicates, and unallocated demand.

5. Pseudocode

```
START
Store students and room information
Create empty room allocation lists
FOR each student: check eligibility and duplicate assignment
Find matching room with available capacity
IF room found, assign; otherwise mark unallocated
Display assignments, duplicate records, and remaining demand
STOP
```

6. Flowchart

START → Store students and rooms → Check eligibility → Check duplicate assignment → Find matching room with capacity → Allocate? → YES: Assign / NO: Unallocated → Display room lists and demand → STOP

7. Python Source Code

```
# University Hostel Room Allocation

# (Room Number, Category, Capacity)
rooms = [
    ("A101", "A", 2),
    ("A102", "A", 1),
    ("B201", "B", 2)
]

# (Student ID, Name, Category, Hostel Approved)
```

```
students = [
    ("S01", "Anil", "A", True),
    ("S02", "Bhavya", "A", True),
    ("S03", "Charan", "B", True),
    ("S04", "Divya", "A", False),
    ("S05", "Esha", "A", True),
    ("S06", "Farhan", "B", True),
    ("S01", "Anil", "A", True)  # Duplicate record for testing
]

allocations = {}
for room_number, category, capacity in rooms:
    allocations[room_number] = []

assigned_ids = set()
unallocated = []
duplicates = []

for student in students:
    student_id, name, category, approved = student

    if student_id in assigned_ids:
        duplicates.append((student_id, name))
        continue

    assigned_ids.add(student_id)

    if not approved:
        unallocated.append((student_id, name, "Not approved"))
        continue

    assigned = False
    for room_number, room_category, capacity in rooms:
        if (category == room_category and
                len(allocations[room_number]) < capacity):
            allocations[room_number].append((student_id, name))
            assigned = True
            break

    if not assigned:
        unallocated.append((student_id, name, "No suitable room available"))

print("HOSTEL ROOM ALLOCATION")
print("----------------------")
for room_number, category, capacity in rooms:
    print("Room:", room_number, "| Category:", category)
    for student_id, name in allocations[room_number]:
        print("  ", student_id, "-", name)
    print("Occupied:", len(allocations[room_number]),
          "/", capacity)
    print("----------------------")

print("DUPLICATE ASSIGNMENTS/RECORDS")
for student_id, name in duplicates:
    print(student_id, "-", name)

print("UNALLOCATED STUDENTS")
for student_id, name, reason in unallocated:
    print(student_id, "-", name, "|", reason)

print("Unallocated Demand:", len(unallocated))
print("Duplicate Records Detected:", len(duplicates))
```

8. Test Cases

Test	Input / Scenario	Condition	Expected Result
1	S01, category A, approved	Matching room has space	Allocated to A101
2	S04, category A, not approved	Approval condition fails	Unallocated
3	S06, category B, approved	Matching room available	Allocated to B201
4	Repeated ID S01	ID already assigned/processed	Duplicate record detected
5	Eligible student but no matching capacity/category	Capacity/category condition fails	Unallocated

9. Sample Output

```
HOSTEL ROOM ALLOCATION
----------------------
Room: A101 | Category: A
   S01 - Anil
   S02 - Bhavya
Occupied: 2 / 2
----------------------
```

```
Room: A102 | Category: A
   S05 - Esha
Occupied: 1 / 1
----------------------
Room: B201 | Category: B
   S03 - Charan
   S06 - Farhan
Occupied: 2 / 2
----------------------
DUPLICATE ASSIGNMENTS/RECORDS
S01 - Anil
UNALLOCATED STUDENTS
S04 - Divya | Not approved
Unallocated Demand: 1
Duplicate Records Detected: 1
```

10. Conclusion

The program allocates approved students to matching rooms while respecting capacity and preventing duplicate assignment. It demonstrates lists, dictionaries, sets, loops, conditions, and allocation logic.